

Mathematical Modelling of Species Differentiation

Classification Criteria for Darevskia Lizards

Solution Report

Based on Morphometric and Pholidosis Characteristics

February 2026

Contents

1	Introduction	2
2	Data Description	2
2.1	Pholidosis Characteristics (Scale Counts)	2
2.2	Morphometric Characteristics (mm)	2
3	Methodology	3
3.1	Performance Metrics	3
3.2	Technical Implementation	3
4	Task 1: Single Parameter Classification	3
4.1	Approach	3
4.2	Solution	4
5	Task 2: Two-Parameter Classification	4
5.1	Approach	4
5.2	Solution	4
6	Task 3: Sex Prediction	4
6.1	Approach	4
6.2	Solution	5
7	Task 4: Subgroup Species Classification	5
7.1	Species #6 and #7	5
7.2	Species #1 and #2	5
7.3	Species #3, #4, and #5	5
8	Task 5: Complete Classification System	6
8.1	Approach	6
8.2	Solution	6
9	Code Implementation	6
9.1	Condition Checker (check_condition.py)	6
9.2	Single Parameter Search (find_one_parameter.py)	7
9.3	Two-Parameter Search (find_pair_parameter.py)	8
9.4	Ratio Search (find_ratio.py)	9
9.5	Three-Parameter Search with Numba (find_triplet_parameters.py)	10
10	Conclusion	12
10.1	Summary of Results	12
10.2	Technical Innovations	12
10.3	Biological Interpretability	13
10.4	References	13

1. Introduction

This report presents a solution to the mathematical modelling problem of species differentiation for lizards belonging to the genus *Darevskia*. The challenge involves developing simple, field-applicable criteria to predict biological species and sex based on measurable morphometric and pholidosis characteristics.

The dataset contains measurements from 564 lizards across 8 species. The goal is to create classification criteria that:

- Achieve the best possible accuracy
- Use simple mathematical operations calculable on an engineering calculator
- Are interpretable by biologists for constructing explanatory theories

The solution employs computational search algorithms to identify optimal thresholds and parameter combinations, while ensuring the final criteria remain simple and practical.

2. Data Description

The dataset contains 23 measured characteristics for each lizard, categorized into pholidosis (scale counts) and morphometric (linear measurements) features.

2.1 Pholidosis Characteristics (Scale Counts)

Parameter	Description
MBS	Medium body scales (dorsal scales at half trunk)
VSN	Ventral scale number on the middle line
CSN	Collar scale number
GSN	Gular scale number (from maxillar angle to collar)
FPNr	Femoral pore number on the right side
SDLr	Subdigital lamellae on 4th toe (right forelimb)
SCSr	Superciliary scales (right side)
SCGr	Superciliary granules (right side)
SMr	Scales between masseteric and supratemporal (right)
MTr	Scales between masseteric and tympanum (right)
PA	Preanal scale number
PTMr	Posttemporal scale number (right side)
aNDSr	Average dorsal scales per abdomen scale (right)

2.2 Morphometric Characteristics (mm)

Parameter	Description
SVL	Snout-vent length (tip of snout to cloaca)
TRL	Trunk length (groin to armpit)
HL	Head length (ventral, snout to collar)
PL	Pileus length (dorsal, snout to parietal/occipital)
ESD	Length of posterior half of pileus
HW	Head width (before tympanic hole)
HH	Head height (near occipital plate)

Parameter	Description
MO	Mouth opening (lateral measurement)
FFL	Total forelimb length
HFL	Total hindlimb length

3. Methodology

The solution approach employs systematic computational search to identify optimal classification criteria. The methodology varies by task complexity:

Task 1 (Single Parameter) Exhaustive threshold search over 1000 steps for each parameter.

Task 2 (Two Parameters) Permutation search across all parameter pairs with 8 logic combinations (AND/OR with $<$ / $>$).

Task 3 (Ratio-based) Search over all possible ratios of two parameters.

Task 4 & 5 (Multi-parameter) Optimized search using Numba JIT compilation and bitwise operations for performance.

3.1 Performance Metrics

Classification accuracy is measured using the confusion matrix approach:

Table 3: Confusion Matrix Format		
	Predicted Positive	Predicted Negative
Actual Positive	a (True Positive)	b (False Negative)
Actual Negative	c (False Positive)	d (True Negative)

Accuracy is calculated as: $\frac{a+d}{a+b+c+d}$

3.2 Technical Implementation

- **Pandas & NumPy:** Data manipulation and vectorized operations
- **Numba:** JIT compilation for performance-critical search loops
- **Bitwise Operations:** Using `numpy.packbits` for memory-efficient boolean storage

4. Task 1: Single Parameter Classification

Objective: Build a criterion that differentiates lizards of species #5 from all other lizards using only the femoral pore number on the right side (FPNr).

4.1 Approach

The algorithm searches for the optimal threshold value for FPNr that maximizes classification accuracy. By testing 1000 threshold values across the parameter's range, we identify the boundary that best separates species #5 from others.

4.2 Solution

Criterion

If $\text{FPNr} < 12$, then the lizard belongs to species #5.

Classification: Species #5 group

Performance Metrics

The criterion achieves optimal separation of species #5 based solely on the femoral pore count, which is a key diagnostic characteristic for this species group.

5. Task 2: Two-Parameter Classification

Objective: Build a criterion that differentiates lizards of species #5 from all other lizards using two variables from the measured characteristics.

5.1 Approach

The algorithm performs an exhaustive search over all possible pairs of parameters ($23 \times 22 = 506$ pairs). For each pair, it tests 8 different logic combinations (AND/OR with various inequality directions) and searches 1000 threshold values for each parameter.

5.2 Solution

Criterion

If $\text{FPNr} < 12$ AND $\text{MBS} < 42$, then the lizard belongs to species #5.

Classification: Species #5 group

Performance Metrics

The two-parameter criterion combines femoral pore number with medium body scales count, providing improved classification accuracy over the single-parameter approach by adding a secondary diagnostic feature.

6. Task 3: Sex Prediction

Objective: Build a criterion that predicts lizards' sex regardless of their biological species, based on morphometric and/or pholidosis characteristics.

6.1 Approach

Following the biological hint that sex may correlate with ratios of linear sizes, the algorithm searches over all possible ratios of two parameters (506 combinations). This approach leverages the expected sexual dimorphism in body proportions.

6.2 Solution

Criterion

If $SVL/PL < 4.62$, then the lizard is Male (M).

Classification: Male (M) / Female (F)

Performance Metrics

The ratio of snout-vent length to pileus length (SVL/PL) provides a robust sex classification criterion. This ratio captures the proportional differences in body structure between male and female lizards across all species.

7. Task 4: Subgroup Species Classification

Objective: Build criteria to differentiate species within subgroups that live together in the same locations.

7.1 Species #6 and #7

Criterion

If $MBS < 48.0002$, then the lizard belongs to species #6.

Classification: Species #6 group

The medium body scales count (MBS) effectively separates these two cohabiting species, with species #6 having generally fewer dorsal scales.

7.2 Species #1 and #2

Criterion

If $MTr/PTMr > 0.618$, then the lizard belongs to species #1.

Classification: Species #1 group

The ratio of masseteric-tympanum scales to posttemporal scales (MTr/PTMr) distinguishes these two species. This ratio captures differences in head scale patterns.

7.3 Species #3, #4, and #5

For this three-species subgroup, a hierarchical classification approach is used:

Classification Hierarchy

Step 1: If $FPNr < 12$ AND $MBS < 42 \rightarrow$ Species #5

Step 2: Else if $GSN > 26 \rightarrow$ Species #4

Step 3: Else if $SCGr/GSN > 0.25 \rightarrow$ Species #3

Classification Strategy

1. First, species #5 is identified by its distinctive FPNr and MBS characteristics
2. Remaining lizards with high gular scale numbers ($\text{GSN} > 26$) are classified as species #4
3. Finally, the ratio of superciliary granules to gular scales separates species #3

8. Task 5: Complete Classification System

Objective: Build a comprehensive set of criteria to predict lizard species across the entire population of 8 species.

8.1 Approach

Using an optimized search algorithm with Numba JIT compilation and bitwise operations, the system searches for three-parameter combinations that best classify each species. The algorithm tests AND/OR logic with various threshold combinations.

8.2 Solution

The complete classification system uses a decision tree with eight rules:

Table 4: Complete Classification Criteria

Group	Criterion	Classification
1	$\text{GSN} < 27.63$ AND $\text{FPNr} > 16.47$ AND $\text{MTr} > 2.21$	Species #1
2	$\text{MBS} > 45.14$ AND $\text{PTMr} > 3.08$ AND $\text{ESD} < 7.05$	Species #2
3	$\text{PA} > 1.00$ AND $\text{aNDSr} > 2.52$ AND $\text{TRL} < 33.67$	Species #3
4	$\text{GSN} > 26.13$ AND $\text{SCGr} < 8.03$	Species #4
5	$\text{MBS} < 41.09$ AND $\text{FPNr} < 11.18$	Species #5
6	$\text{VSN} > 25.06$ AND $\text{SDLr} < 21.18$ AND $\text{SCGr} < 9.18$	Species #6
7	$\text{VSN} > 26.10$ AND $\text{SDLr} < 21.18$ AND $\text{SCGr} > 9.18$	Species #7
8	$\text{MBS} < 47.27$ AND $\text{ESD} < 7.30$ AND $\text{FFL} > 19.55$	Species #8

Classification Strategy

The criteria are designed to be evaluated in sequence. Each rule uses a combination of 2–3 parameters with AND logic to maximize specificity. The parameters were selected to maximize inter-species separation while maintaining biological interpretability.

9. Code Implementation

The solution is implemented through a modular Python codebase with specialized scripts for each task type.

9.1 Condition Checker (check_condition.py)

This utility script evaluates any given classification condition and reports accuracy metrics:

```

1 import pandas as pd
2
3 df = pd.read_csv("lizards_data.csv", sep=',')
4 df = df.sort_values(by='Species_num',
5                     ascending=True).reset_index(drop=True)
6
7 GROUP = 7 # Target species to evaluate
8
9 def condition(row):
10     return row["VSN"] > 26.10 and row["SDLr"] < 21.18 and
11         row["SCGr"] > 9.18
12
13 # Count total in target group
14 total = sum(1 for i in range(len(df)) if df.loc[i, "Species_num"] ==
15            GROUP)
16
17 # Evaluate predictions
18 true_predict = 0
19 false_predict = 0
20 for i in range(len(df)):
21     row = df.loc[i]
22     if condition(row):
23         if row["Species_num"] == GROUP:
24             true_predict += 1
25         else:
26             false_predict += 1
27
28 print(f"Total: {total}")
29 print(f"True: {true_predict}")
30 print(f"False: {false_predict}")
31 print(f"Acc: {1 - (total - true_predict + false_predict) / len(df)}")

```

9.2 Single Parameter Search (find_one_parameter.py)

Exhaustive search for optimal single-parameter thresholds:

```

1 import pandas as pd
2 import numpy as np
3
4 df = pd.read_csv("lizards_data.csv", sep=',')
5 GROUP = 5
6
7 PARAMETERS = ["MBS", "VSN", "CSN", "GSN", "FPNr", "SDLr", "SCSr",
8              "SCGr", "SMr", "MTr", "PA", "PTMr", "aANDSr", "SVL",
9              "TRL", "HL", "PL", "ESD", "HW", "HH", "MO", "FFL",
10              "HFL"]
11
12 actual_species = (df["Species_num"] == GROUP).to_numpy()
13 super_results = []
14
15 for param in PARAMETERS:
16     values = df[param].to_numpy()
17     min_v, max_v = np.min(values), np.max(values)
18     thresholds = np.linspace(min_v, max_v, 1000)
19
20     best_acc, best_thresh = -1, 0

```



```

20     for thresh in thresholds:
21         predict_is_1 = values < thresh
22         correct = (predict_is_1 == actual_species)
23         acc = np.mean(correct)
24         if acc > best_acc:
25             best_acc, best_thresh = acc, thresh
26
27     super_results.append([param, best_thresh, best_acc])
28
29 super_results.sort(key=lambda x: x[2], reverse=True)
30 print(f"Best: {super_results[0][0]}, Threshold:
      {super_results[0][1]:.2f}")

```

9.3 Two-Parameter Search (find_pair_parameter.py)

Vectorized search for optimal parameter pairs with combined logic:

```

1  import pandas as pd
2  import numpy as np
3  import itertools
4
5  TARGET_SPECIES = [5]
6  BACKGROUND_SPECIES = [1, 2, 3, 4, 5, 6, 7, 8]
7  STEPS = 1000
8
9  df = pd.read_csv("lizards_data.csv", sep=',')
10 PARAMETERS = ["MBS", "VSN", "CSN", "GSN", "FPNr", "SDLr", ...]
11
12 # Filter relevant data
13 relevant_mask = df["Species_num"].isin(TARGET_SPECIES +
14     BACKGROUND_SPECIES)
15 df_clean = df[relevant_mask].copy()
16 arr_of_true = df_clean["Species_num"].isin(TARGET_SPECIES).to_numpy()
17
18 # Logic configurations
19 logic_configs = [
20     ("p1 > t1 AND p2 > t2", 1, '&', 1),
21     ("p1 < t1 AND p2 < t2", -1, '&', -1),
22     ("p1 < t1 AND p2 > t2", -1, '&', 1),
23     ("p1 > t1 AND p2 < t2", 1, '&', -1),
24     ("p1 > t1 OR p2 > t2", 1, '|', 1),
25     ("p1 < t1 OR p2 < t2", -1, '|', -1),
26     ("p1 < t1 OR p2 > t2", -1, '|', 1),
27     ("p1 > t1 OR p2 < t2", 1, '|', -1),
28 ]
29
30 super_results = []
31 for p1_name, p2_name in itertools.permutations(PARAMETERS, 2):
32     v1 = df_clean[p1_name].to_numpy()
33     v2 = df_clean[p2_name].to_numpy()
34
35     t1_vals = np.linspace(v1.min(), v1.max(), STEPS)
36     t2_vals = np.linspace(v2.min(), v2.max(), STEPS)
37
38     # Vectorized comparison matrices
39     m1_g = v1[:, None] > t1_vals[None, :]

```

```

40     m2_g = v2[:, None] > t2_vals[None, :]
41     m2_l = v2[:, None] < t2_vals[None, :]
42
43     arr_of_true_col = arr_of_true[:, None]
44
45     for label, op1, logic, op2 in logic_configs:
46         mat1 = m1_g if op1 == 1 else m1_l
47         mat2 = m2_g if op2 == 1 else m2_l
48
49         best_local_acc, best_t1, best_t2 = -1, 0, 0
50
51         for i in range(STEPS):
52             col1 = mat1[:, i:i+1]
53             preds = col1 & mat2 if logic == '&' else col1 | mat2
54             accuracies = (preds == arr_of_true_col).mean(axis=0)
55             max_idx = np.argmax(accuracies)
56
57             if accuracies[max_idx] > best_local_acc:
58                 best_local_acc = accuracies[max_idx]
59                 best_t1, best_t2 = t1_vals[i], t2_vals[max_idx]
60
61         super_results.append({
62             "Params": [p1_name, p2_name],
63             "Logic": label,
64             "T1": best_t1, "T2": best_t2,
65             "Accuracy": best_local_acc
66         })
67
68     super_results.sort(key=lambda x: x["Accuracy"], reverse=True)

```

9.4 Ratio Search (find_ratio.py)

Search for optimal parameter ratios for sex classification:

```

1  import pandas as pd
2  import numpy as np
3
4  df = pd.read_csv("lizards_data.csv", sep=',')
5  GROUP = 8
6  STEPS = 100
7  PARAMETERS = ["MBS", "VSN", "CSN", "GSN", "FPNr", "SDLr", ...]
8
9  actual_species = (df["Species_num"] == GROUP).to_numpy()
10 super_results = []
11
12 for p1 in PARAMETERS:
13     for p2 in PARAMETERS:
14         if p1 == p2:
15             continue
16         ratios = df[p1].to_numpy() / df[p2].to_numpy()
17         min_v, max_v = np.min(ratios), np.max(ratios)
18         thresholds = np.linspace(min_v, max_v, STEPS)
19
20         best_acc, best_thresh = -1, 0
21         for thresh in thresholds:
22             predict_is_correct = ratios > thresh
23             correct = (predict_is_correct == actual_species)

```

```

24         acc = np.mean(correct)
25         if acc > best_acc:
26             best_acc, best_thresh = acc, thresh
27
28         super_results.append([[p1, p2], best_thresh, best_acc])
29
30 super_results.sort(key=lambda x: x[2], reverse=True)
31 print(f"Best ratio:
      {super_results[0][0][0]}/{super_results[0][0][1]}")

```

9.5 Three-Parameter Search with Numba (find_triplet_parameters.py)

High-performance search using Numba JIT and bitwise operations:

```

1  import pandas as pd
2  import numpy as np
3  import itertools
4  from time import time
5  from numba import njit, prange
6
7  TARGET_SPECIES = [8]
8  BACKGROUND_SPECIES = [1, 2, 3, 4, 5, 6, 7, 8]
9  STEPS = 10
10
11 df = pd.read_csv("lizards_data.csv", sep=',')
12 PARAMETERS = ["MBS", "VSN", "CSN", "GSN", "FPNr", "SDLr", ...]
13
14 df_clean = df[df["Species_num"].isin(TARGET_SPECIES +
15                                     BACKGROUND_SPECIES)].copy()
16 arr_of_true = df_clean["Species_num"].isin(
17     TARGET_SPECIES).to_numpy().astype(bool)
18
19 # Pack boolean array into bits for memory efficiency
20 arr_of_true_packed = np.packbits(arr_of_true)
21 real_n_samples = len(arr_of_true)
22 n_bytes = len(arr_of_true_packed)
23
24 # Precompute threshold masks
25 n_params = len(PARAMETERS)
26 masks_packed = np.zeros((n_params, STEPS, n_bytes), dtype=np.uint8)
27 thresholds_all = np.zeros((n_params, STEPS), dtype=np.float32)
28
29 for i, param in enumerate(PARAMETERS):
30     vals = df_clean[param].to_numpy()
31     th = np.linspace(vals.min(), vals.max(), STEPS)
32     thresholds_all[i, :] = th
33     bool_matrix = (vals[:, None] > th[None, :])
34     masks_packed[i, :, :] = np.packbits(bool_matrix.T, axis=1)
35
36 # Population count lookup table
37 POP_COUNT_TABLE = np.array([bin(x).count('1') for x in range(256)],
38                             dtype=np.uint8)
39
40 @njit(fastmath=True, parallel=True)
41 def fast_search(masks, y_packed, pop_table, steps, n_params,
42                n_bytes, real_n_samples):
43     results = np.zeros((n_params, n_params, n_params, 6),

```

```

dtype=np.float32)
41
42 remainder = real_n_samples % 8
43 last_byte_mask = np.uint8(0xFF << (8 - remainder)) if remainder
    > 0 else np.uint8(255)
44
45 for p1 in prange(n_params):
46     for p2 in range(p1 + 1, n_params):
47         for p3 in range(p2 + 1, n_params):
48             m1_pack = masks[p1]
49             m2_pack = masks[p2]
50             m3_pack = masks[p3]
51
52             best_acc = -1.0
53             curr_res = np.zeros(6, dtype=np.float32)
54
55             for t1 in range(steps):
56                 r1 = m1_pack[t1]
57                 for t2 in range(steps):
58                     r2 = m2_pack[t2]
59                     for t3 in range(steps):
60                         r3 = m3_pack[t3]
61
62                         for s1 in range(2):
63                             for s2 in range(2):
64                                 for s3 in range(2):
65                                     sign_code = (s1 << 2) | (s2
66                                         << 1) | s3
67                                     hits_and, hits_or = 0, 0
68
69                                     for b in range(n_bytes):
70                                         v1 = r1[b] if s1 else
71                                             ~r1[b]
72                                         v2 = r2[b] if s2 else
73                                             ~r2[b]
74                                         v3 = r3[b] if s3 else
75                                             ~r3[b]
76                                         yt = y_packed[b]
77
78                                         mask_p = last_byte_mask
79                                         if b == n_bytes - 1
80                                             else np.uint8(255)
81
82                                         # Bitwise AND logic
83                                         match_and = (~(v1 & v2
84                                             & v3) ^ yt)) & mask_p
85                                         hits_and +=
86                                             pop_table[match_and]
87
88                                         # Bitwise OR logic
89                                         match_or = (~(v1 | v2 |
90                                             v3) ^ yt)) & mask_p
91                                         hits_or +=
92                                             pop_table[match_or]
93
94                                         acc_and = hits_and /
95                                             real_n_samples
96                                         if acc_and > best_acc:

```

```

86         best_acc = acc_and
87         curr_res[:] = [acc_and,
88                        t1, t2, t3,
89                        sign_code, 0.0]
90
91         acc_or = hits_or /
92         real_n_samples
93         if acc_or > best_acc:
94             best_acc = acc_or
95             curr_res[:] = [acc_or,
96                            t1, t2, t3,
97                            sign_code, 1.0]
98
99         results[p1, p2, p3] = curr_res
100     return results
101
102 # Execute search
103 start_time = time()
104 res_cube = fast_search(masks_packed, arr_of_true_packed,
105                        POP_COUNT_TABLE,
106                        STEPS, n_params, n_bytes, real_n_samples)
107 print(f"Search completed in {time() - start_time:.4f} seconds")

```

10. Conclusion

This solution provides a comprehensive set of classification criteria for differentiating lizard species and predicting sex based on morphometric and pholidosis measurements.

10.1 Summary of Results

Table 5: Summary of Classification Criteria

Task	Criterion Type	Key Parameters
1	Single parameter	FPNr < 12
2	Two parameters (AND)	FPNr < 12 AND MBS < 42
3	Ratio	SVL/PL < 4.62
4a	Single parameter	MBS < 48.0002
4b	Ratio	MTr/PTMr > 0.618
4c	Hierarchical (3 rules)	FPNr/MBS, GSN, SCGr/GSN
5	Decision tree (8 rules)	Various 2–3 parameter combinations

10.2 Technical Innovations

- **Vectorized Search:** NumPy array operations enable efficient threshold searching across thousands of combinations
- **Bitwise Optimization:** Using `numpy.packbits` reduces memory usage and enables fast parallel comparison
- **Numba JIT:** Just-in-time compilation accelerates the search by 10–100×
- **Parallel Processing:** The `prange` decorator enables multi-core execution

10.3 Biological Interpretability

All criteria use simple mathematical operations (comparisons, ratios, AND/OR logic) that can be computed with an engineering calculator. The selected parameters have biological relevance:

- **FPNr (Femoral Pores):** Sexually dimorphic feature, important for species identification
- **MBS (Dorsal Scales):** Reflects body armor and ecological adaptation
- **GSN (Gular Scales):** Throat region characteristic, species-specific pattern
- **Body Ratios:** Capture proportional differences in body structure

10.4 References

1. NumPy Documentation: `numpy.packbits` — numpy.org
2. Real Python: Python Bitwise Operators — realpython.com
3. Numba Documentation: Parallel JIT — numba.pydata.org